COMSATS University Islamabad
Vehari Campus

SOFTWARE DESIGN DESCRIPTION

(SDD Document)

for

MyCampusRide

Version 1.0

By

Ahtisham Tahir **CIIT/FA22-BCS-012/VHR**
Khizar hayat **CIIT/FA22-BCS-023/VHR**

Supervisor
Mr. Rizwan Ali

Bachelor of Science in Computer Science (2022-2026)

Table of Contents

Revision History 1
1. Introduction 1
2. Design methodology and software process model 1
3. System overview 1
3.1. Architectural design 2
3.1.1 System Architecture Description 2
3.1.2 System Architecture Diagram 3
3.1.3 Modular Decomposition 3
3.1.4 Module Interactions 3
3.2. Process flow Representation 3
3.2.1 Activity Diagram(s) for Major Processes 3
4. Design models 5
4.1 Class Diagram 5
4.2 Sequence Diagram 6
4.3 State Transition Diagram 9
4.4 Data Flow Diagram (DFD Level 0–2) 10
5. Data design 15
5.1 Data Dictionary 16
6. Algorithm & Implementation 17
7. Software requirements traceability matrix 18
8. Human interface design 18
8.1 Screen images 19
8.2 Screen objects and actions 23
9. References 24

Revision History

Name	Date	Review for changes	Version

Application Evaluation History

Comments (by committee)	Action Taken

Supervised By

Sir Rizwan Ali

Signature_____

1. Introduction

MyCampusRide is an end-to-end campus bus transportation management system, designed to make the process of managing bus routes, tracking the location of buses in real time, and facilitating communication between administrators, drivers, and students smoother. It solves some of the key pain areas related to managing transportation infrastructure efficiently that most educational institutions face.

MyCampusRide primarily focuses on providing a central platform for administrators in which they will be able to manage buses, routes, and users while allowing drivers to manage their trips, update their locations, track buses, and have the students receive notifications about their transportation. The system will use modern web technologies to present a responsive and intuitive interface for the mentioned user roles.

This Software Design Document provides a complete description of the system architecture, design methodology, and implementation details of the MyCampusRide system. The document is a technical reference to developers, stakeholders, and future maintainers of the system. It covers the architectural design of the system, process flows, data models, algorithms, and human interface design.

2. Design methodology and software process model

This system is developed using the Agile software development methodology. This iterative approach has been chosen because it allows flexibility in response to changing requirements. It provides an opportunity for continued feedback by the clients, and allows incremental delivery of functional features.

The architecture design of the MyCampusRide follows the Model View Controller model, allowing for a organized and clear separation of concerns between data management, business logic, and presentation layers. The frontend implements a component based architecture using React.js library, while the backend follows RESTful API design principles with Express.js framework. This architectural choice guarantees the maintainability, testability, and scalability of the codebase.

Version control was effected with Git, while following a feature-branch workflow. All the pull requests had to go through code reviews for quality and standards check. Continuous integration practices were put in place to automate tests, ensuring that new changes did not break existing functionality.

3. System overview

MyCampusRide follows the Agile software development methodology. This iterative approach has been chosen because it provides flexibility in responding to changes in the requirements, continuous feedback by the clients, and incremental delivery of functional features in flexible way.

The system architecture is designed using the Model-View-Controller pattern, implementing clean separations of concerns between data management, business logic, and presentation layers. The frontend implements a component-based architecture in React.js, while the backend adheres to RESTful API design principles using Express.js framework. This choice of architecture provides for the maintainability, testability, and scalability of the codebase.

Version control was done using Git, using the feature-branch workflow. The code quality was further ensured using code reviews on all the pull requests and followed the set coding standards. Continuous integration practices were engaged to automate tests, ensuring new changes did not break the functionality.

3.1. Architectural design

3.1.1 System Architecture Description

MyCampusRide is a full-stack, web-based application intended for use in the management of transport operations within campus settings. The system has three major user roles: administrators, drivers, and students, and it allows functional access based on the needs of each of these user types.

Frontend: The application is built using React.js, with Material-UI applied to have consistent and responsive user interface components. Vite acts as a build tool with the best possible development experience and high performance in production. The application will be using client-side routing implemented with React Router and state management with React Context API.

Backend: The application is being developed using Node.js with the Express.js framework, providing a robust and scalable server-side infrastructure. MongoDB is used as the database management system in order to have flexibility in data modeling and very good performance for document-oriented data structures present within the application. Features of the system include user authentication and authorization at a role-based level of access control, full bus and route management, real-time bus tracking, with the GPS coordinates of each vehicle; a notification system for communicating between users, driver approval workflow; and student fee management. In every user role, there will be a dedicated dashboard, showing functionalities and information relevant to it.

3.1.2 System Architecture Diagram

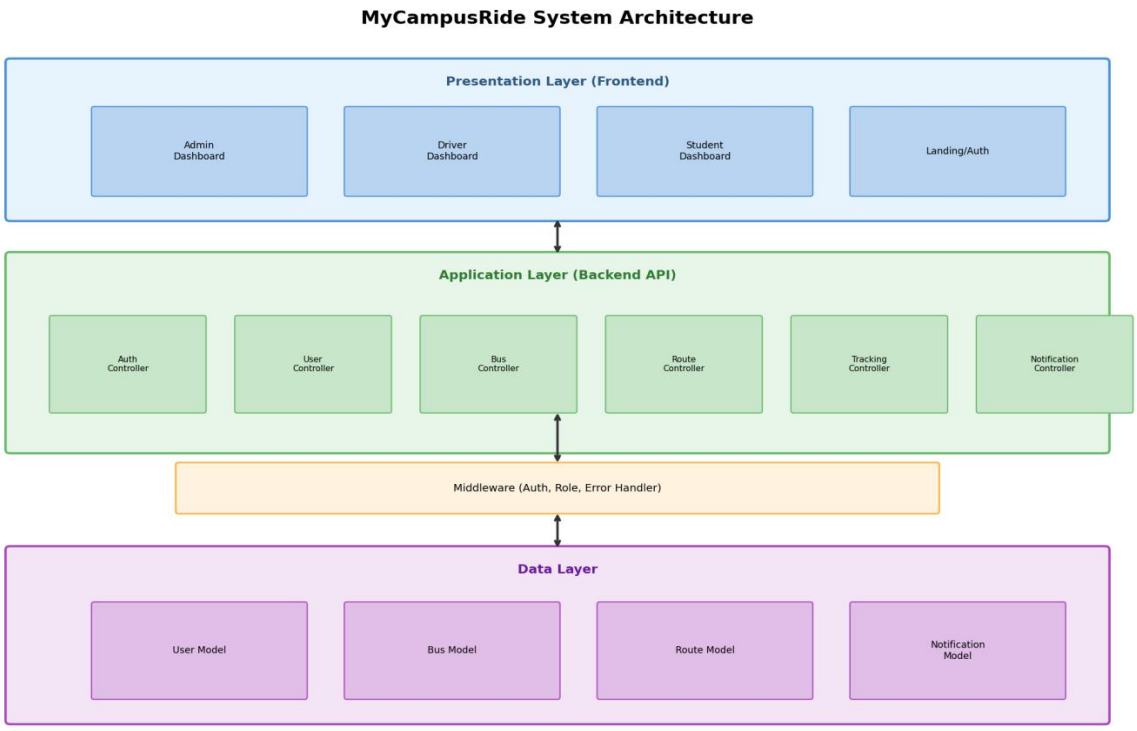


Figure 4.1: MyCampusRide System Architecture

3.1.3 Modular Decomposition

The system is decomposed into the following major modules:

Authentication Module: This module handles user registration, login, token generation through JWT, password hashing with bcrypt, and session management. It provides a secure access control to protected resources.

User Management Module: It manages all user administrative functions like creating, updating, and deleting, apart from user status changes. It also handles the driver approval workflow and generation of user statistics.

Bus Management Module: responsible for doing CRUD on bus entities, assigning drivers and routes to them, managing state transitions, and calculating statistics about buses.

Route Management Module: It manages the transportation routes consisting of stops along with their coordinates, timings, and fees. It manages the activation of a route, assigns buses to a certain route, and also deals with route statistics.

Tracking Module: Tracking of buses in real-time using GPS coordinates; trip lifecycle management for starting and stopping working routines; and student location updates.

Notification Module: This module provides an organized channel of communication among the users of the system, using both target and broadcast notifications. It manages notifications through their entire life cycle, which includes creating, reading, and deleting.

3.1.4 Module Interactions

These modules interact through well-defined interfaces and shared data models. The Authentication Module acts like a gateway for every protected operation, checking the validity of tokens and attaching user information to requests. Other modules depend on this authenticated information to enforce the rules of authorization.

The Bus Management Module interacts with User Management for driver assignments and with Route Management for route associations. In the creation or update of a bus, the system validates if the driver in charge is active and not already allocated to another bus. The Tracking Module works in collaboration with Bus Management for updating the status and location information of the buses. When the trip starts or completes, it also triggers the Notification Module to send notifications to the students. This interaction ensures timely information is provided to the students for their transportation.

3.2. Process flow Representation

3.2.1 Activity Diagram(s) for Major Processes

The following are some activity diagrams that depict major processes in the MyCampusRide system. Each of these diagrams shows the flow of activities from initiation through to completion, the decision points, and parallel activities.

Bus Tracking Process:

The process of tracking a bus begins by opening an application and starting the trip initiated by the driver. The system checks if that driver has an assigned bus, changes the status of the bus to "on trip", and sends automated notifications to the subscribers of that particular route.

During the journey, the location of the driver gets updated until the completion of a particular trip.

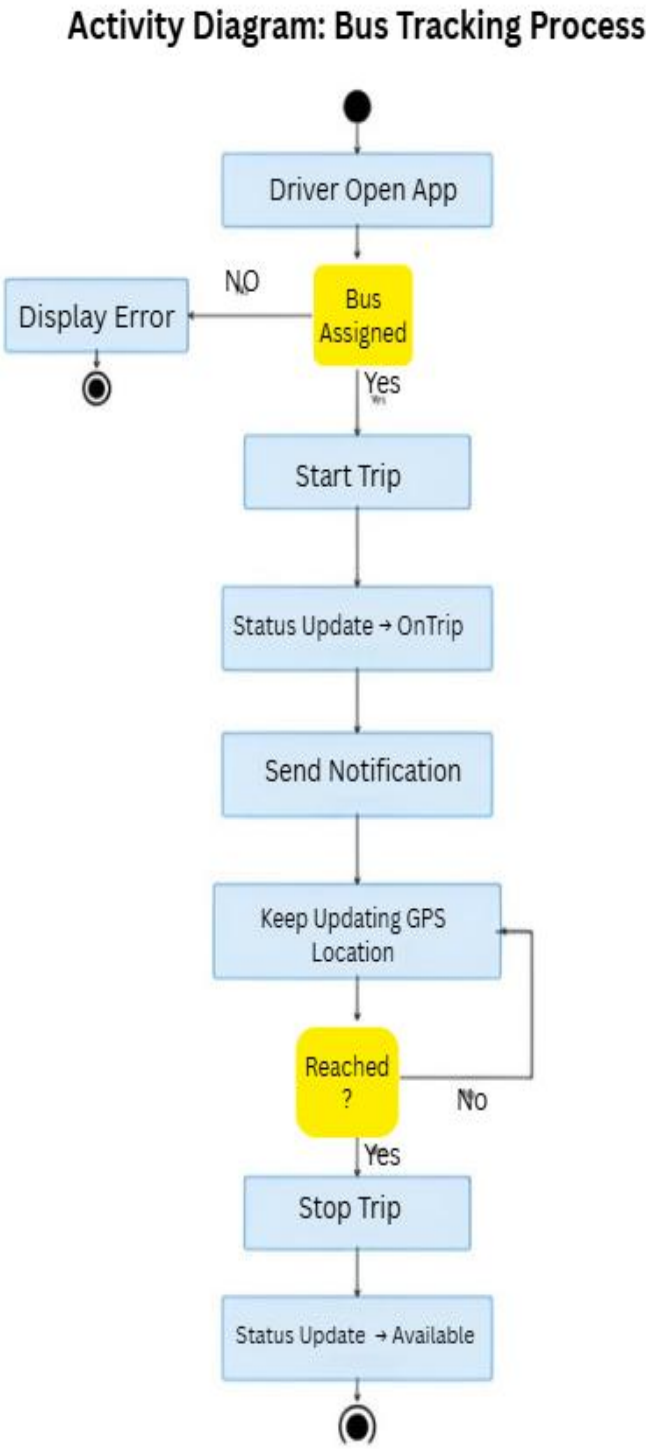


Figure 5.1: Activity Diagram - Bus Tracking Process

Route Management Process:

The route management process allows administrators to create and configure transportation routes. The admin enters route details, pickup times, and fees. The system validates the data

for uniqueness and correctness before saving to the database. After validation, buses can be assigned to the created route.

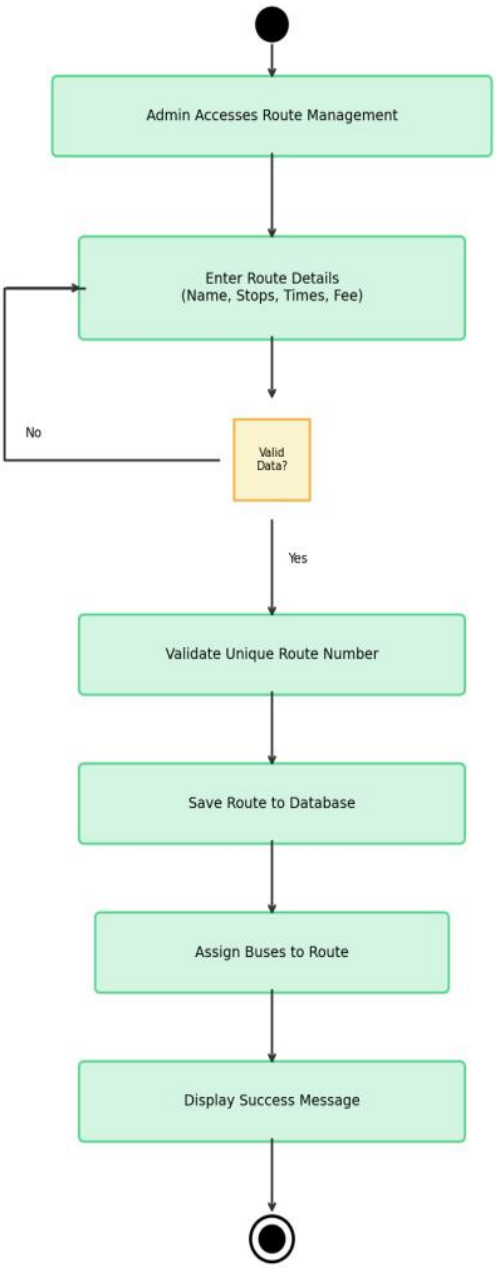


Figure 5.2: Activity Diagram - Route Management Process

4. Design models

4.1 Class Diagram

This class diagram shows the static structure of the MyCampusRide system, including main data entities, their attributes, methods, and relationships. The system has been designed using

four primary models, namely, User, Bus, Route, and Notification. Each route contains a Stop subdocument.

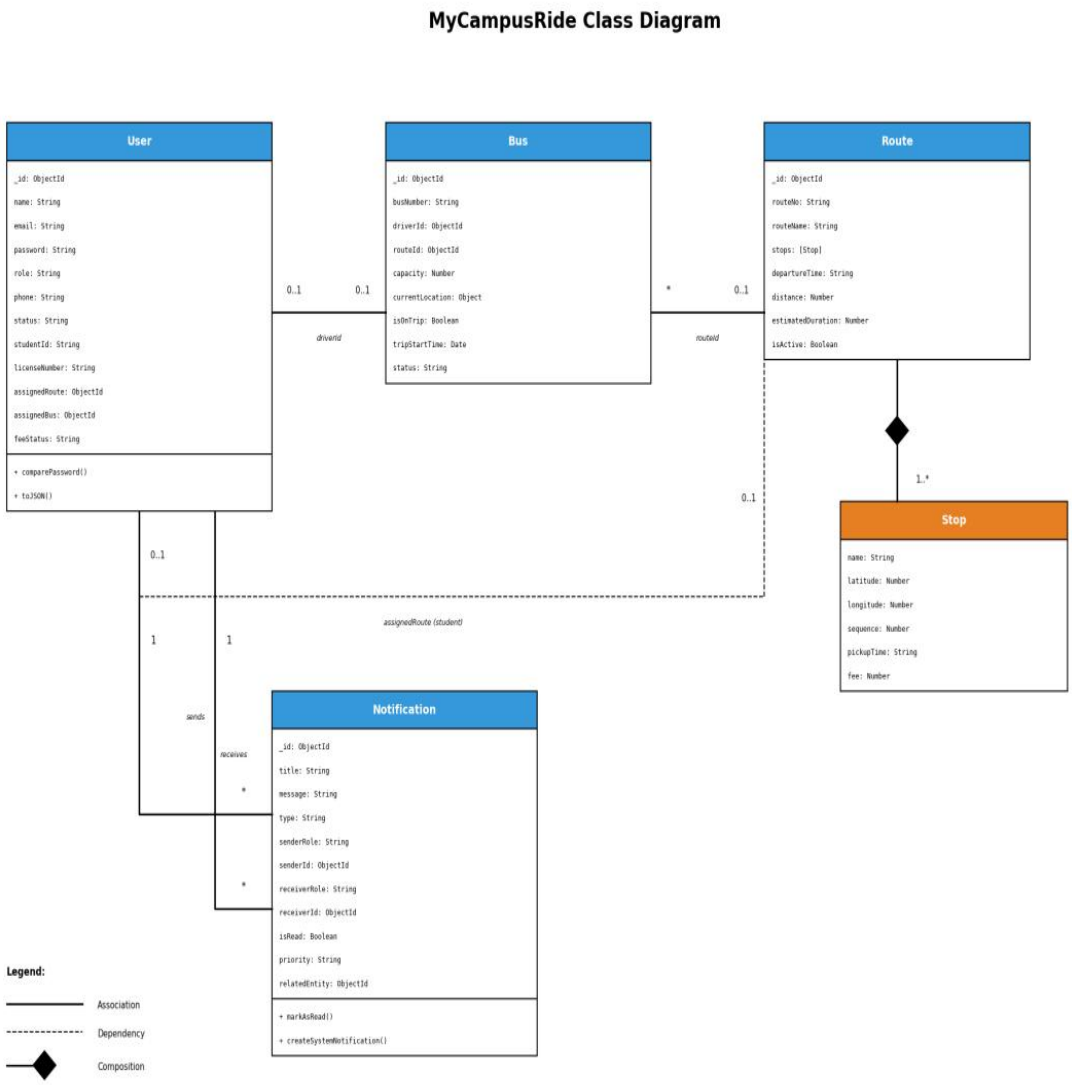
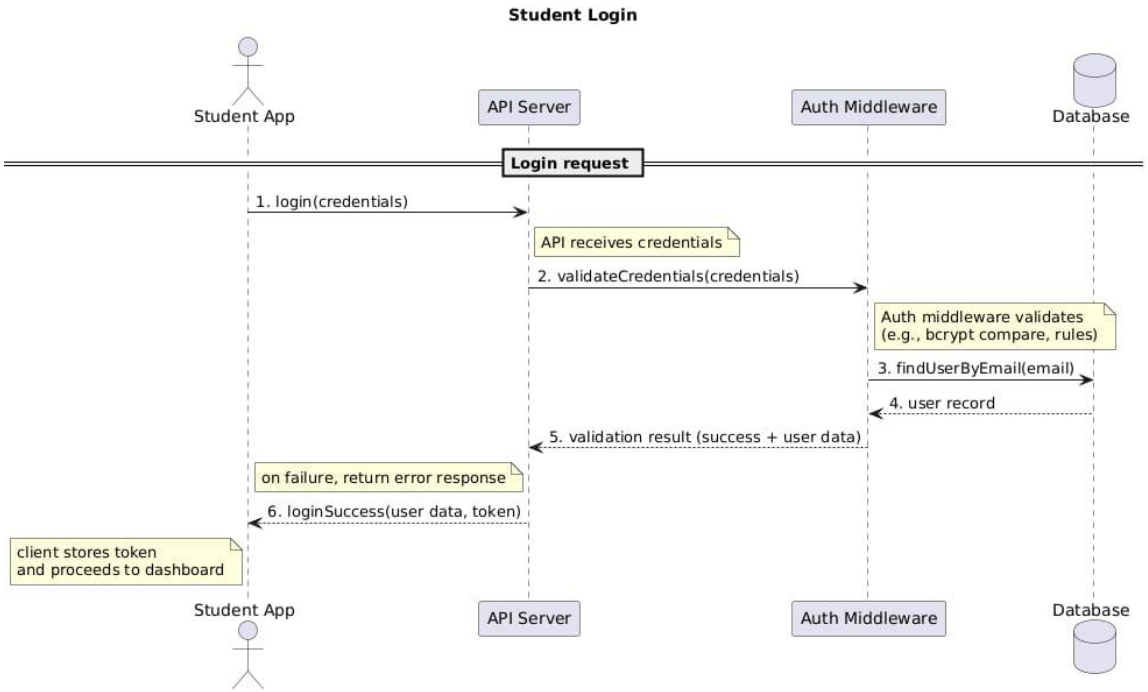
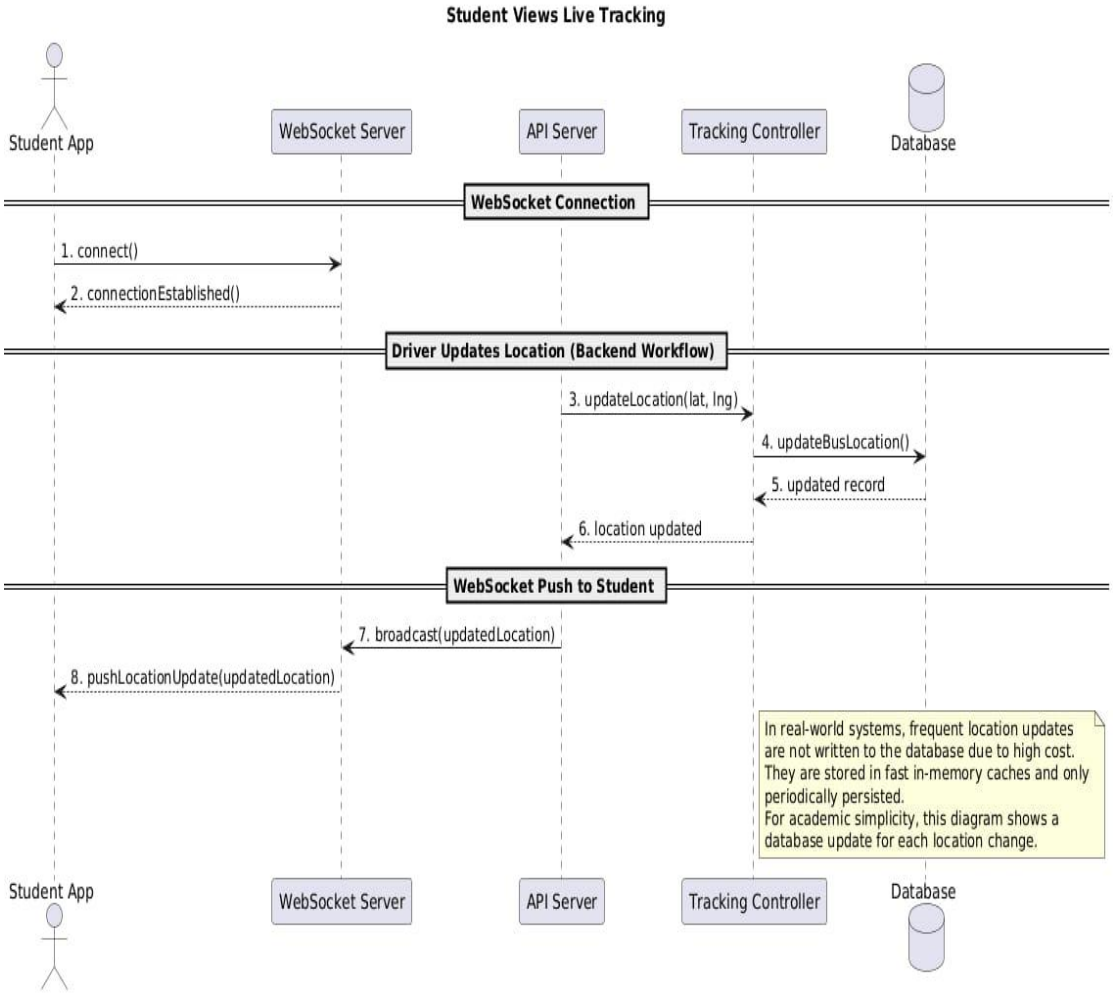


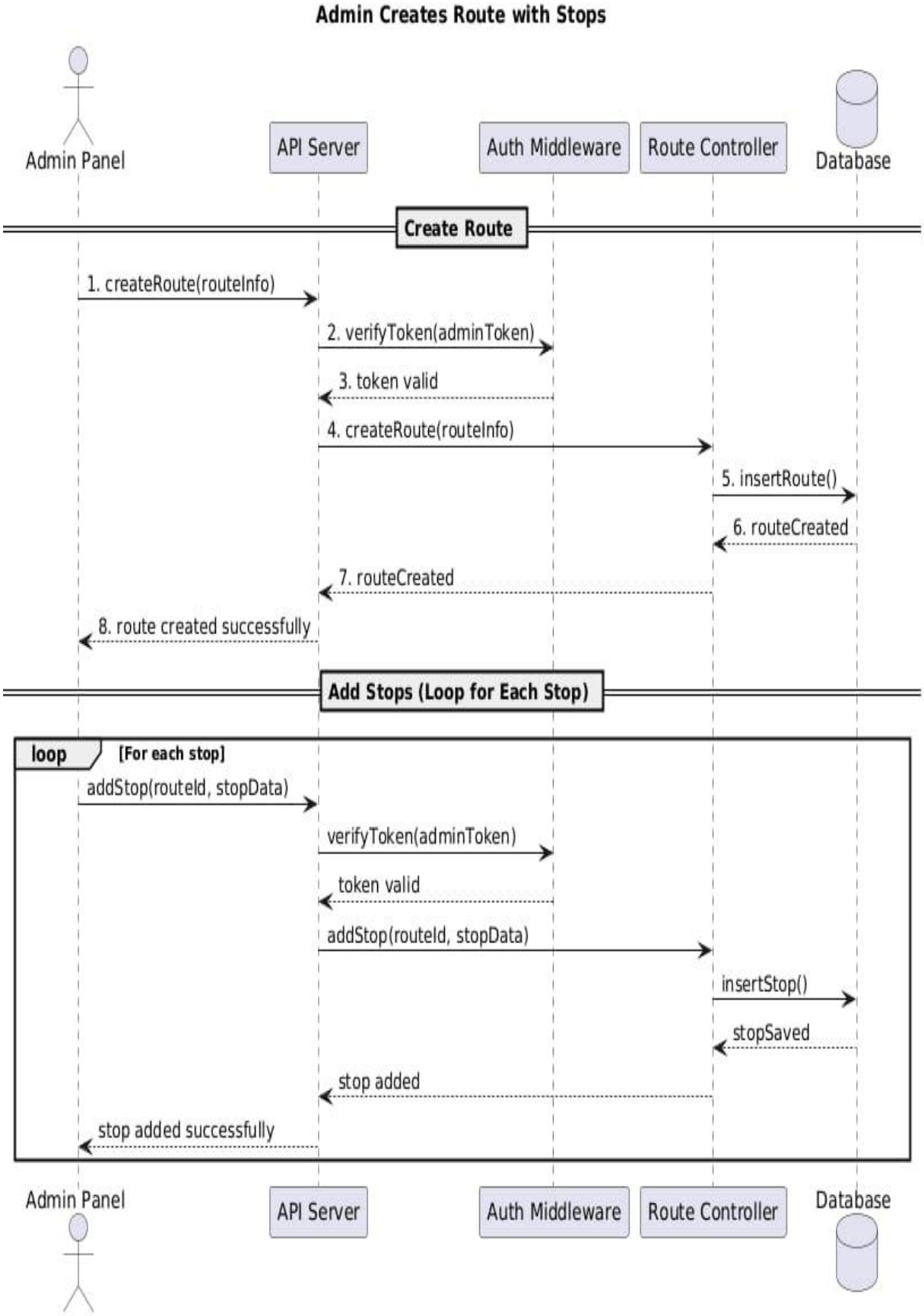
Figure 6.1: MyCampusRide Class Diagram

4.2 Sequence Diagram

This sequence diagram shows how system components interact in the real-time bus tracking operation. It states the flow of messages from the Driver dashboard using the API Server, Authentication Middleware, Tracking Controller, and Database that show how live location updates are processed and stored.


Student Login
Student App
API Server
Auth Middleware
Database
Login request
1. login(credentials)
API receives credentials
2. validateCredentials(credentials)
Auth middleware validates
(e.g., bcrypt compare, rules)
3. findUserByEmail(email)
4. user record
5. validation result (success + user data)
on failure, return error response
6. loginSuccess(user data, token)
client stores token
and proceeds to dashboard
Student App
API Server
Auth Middleware
Database


Student Views Live Tracking
Student App
WebSocket Server
API Server
Tracking Controller
Database
WebSocket Connection
1. connect()
2. connectionEstablished()
Driver Updates Location (Backend Workflow)
3. updateLocation(lat, lng)
4. updateBusLocation()
5. updated record
6. location updated
WebSocket Push to Student
7. broadcast(updatedLocation)
8. pushLocationUpdate(updatedLocation)
In real-world systems, frequent location updates
are not written to the database due to high cost.
They are stored in fast in-memory caches and only
periodically persisted.
For academic simplicity, this diagram shows a
database update for each location change.
Student App
WebSocket Server
API Server
Tracking Controller
Database


Admin Creates Route with Stops
Admin Panel
API Server
Auth Middleware
Route Controller
Database
Create Route
1. createRoute(routeInfo)
2. verifyToken(adminToken)
3. token valid
4. createRoute(routeInfo)
5. insertRoute()
6. routeCreated
7. routeCreated
8. route created successfully
Add Stops (Loop for Each Stop)
loop
[For each stop]
addStop(routeId, stopData)
verifyToken(adminToken)
token valid
addStop(routeId, stopData)
insertStop()
stopSaved
stop added
stop added successfully
Admin Panel
API Server
Auth Middleware
Route Controller
Database

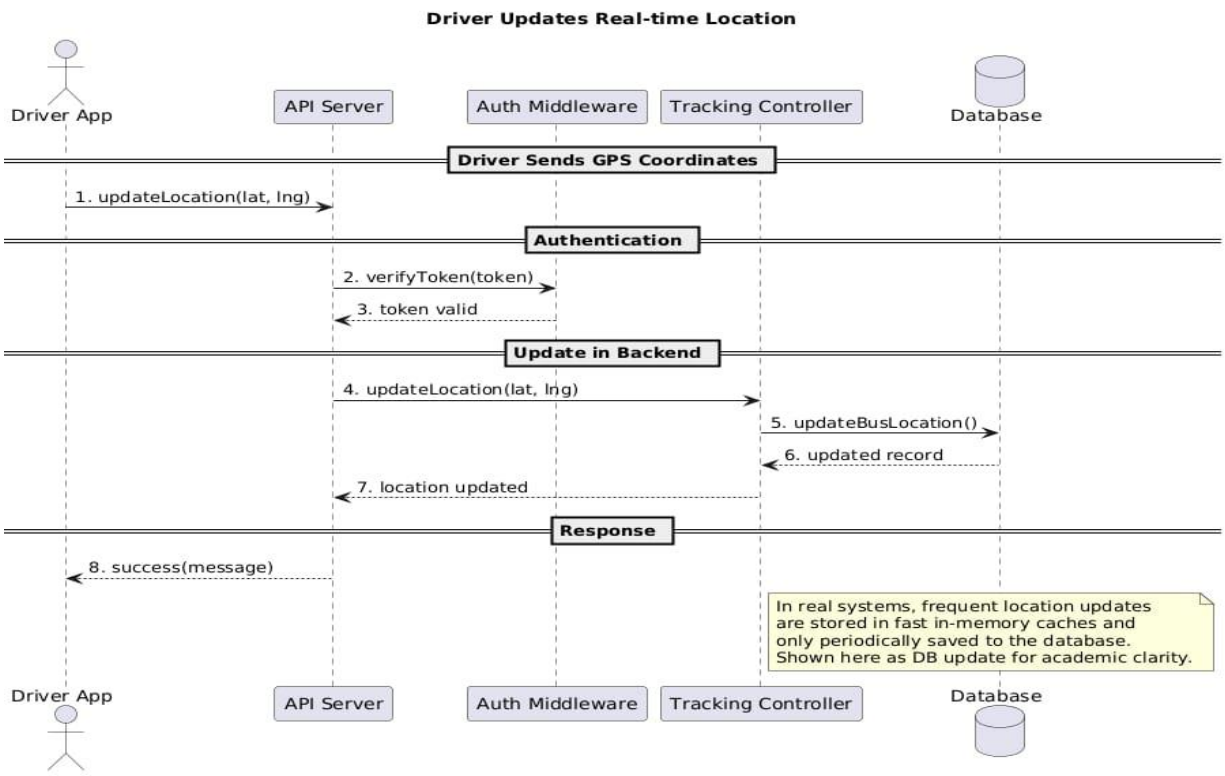


Figure 6.2: Sequence Diagram - Real-time Bus Tracking

4.3 State Transition Diagram

The state transition diagram demonstrates all the various states through which a bus can be throughout its lifecycle and what events trigger transitions between these states. A bus starts in the "Available" state and can transition to "On Trip", "Maintenance", or "Out of Service" states based on the driver actions or admin.

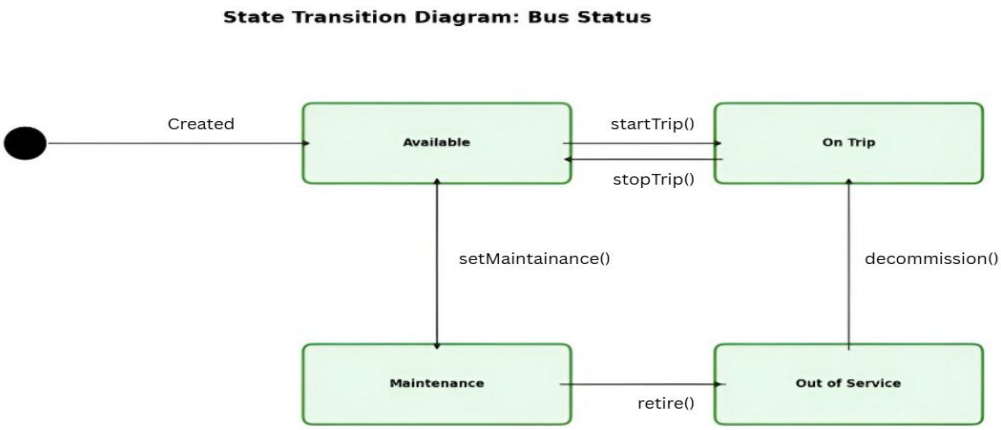


Figure 6.3: State Transition Diagram - Bus Status

4.4 Data Flow Diagram (DFD Level 0–2)

DFD Level 0 - Context Diagram:

The DFD Level 0 which is also known as Context Diagram, for MyCampus Ride represents the system as a central process that is interacting with Admin, Driver, Student, and Database. Admin sends Login Request, Update User Operations, Update the Routes, and Fee Status, receiving Auth Responses and Operation Results. Driver inputs include Login Request, Tracking Request, Trip Status, and GPS Coordinates, getting Auth Response and Trip Updates. Student inputs include Login Request and Tracking Request, receiving Auth Response and Trip Updates. This high-level view defines system boundaries and key data flows for campus transport management.

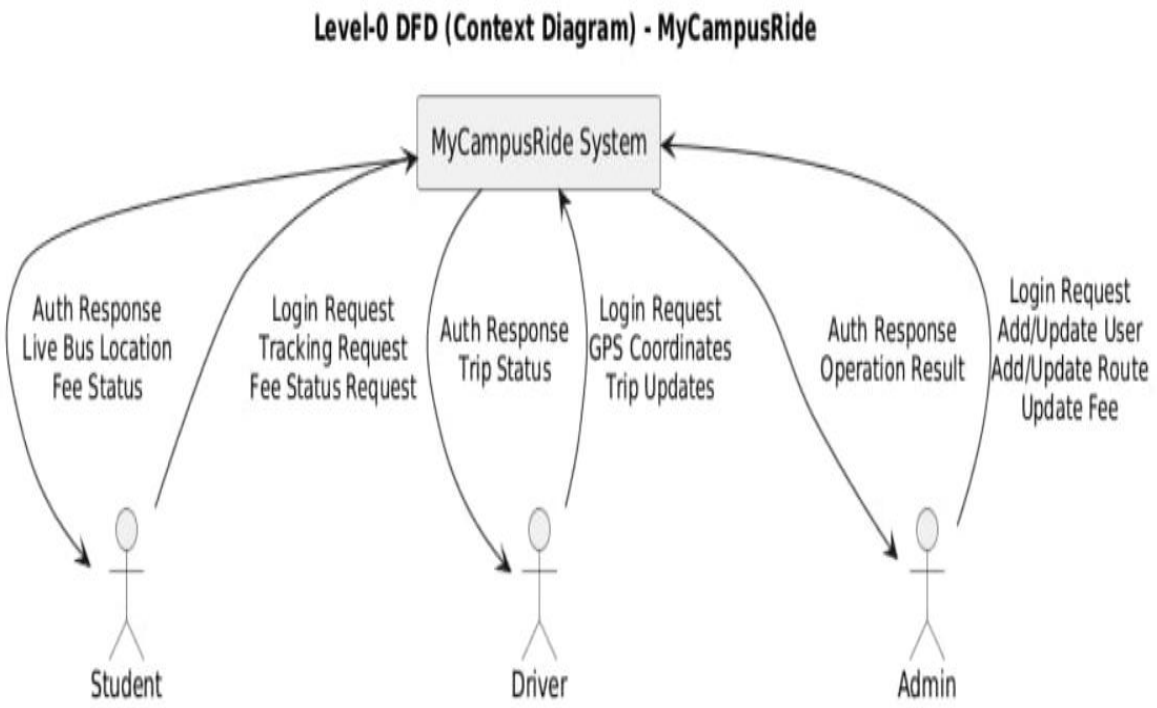


Figure 6.4: DFD Level 0 - Context Diagram

DFD Level 1:

The Level 1 DFD breaks down the major system into six major processes, which are: User Management, Bus Management, Route Management, Tracking, Notification, and Authentication. The data stores for Users, Buses, Routes, and Notifications are displayed, showing the data flows between processes and stores.

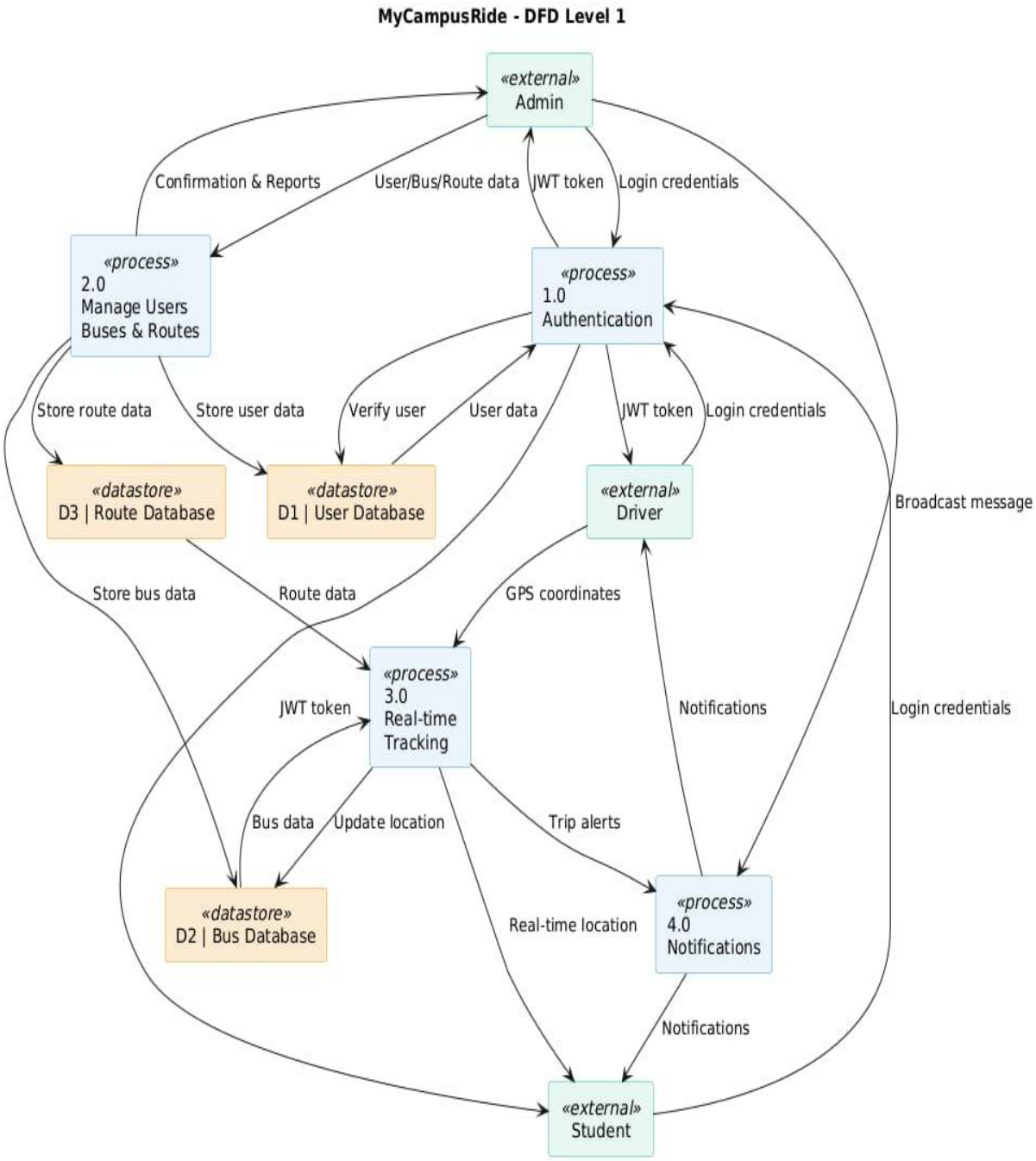
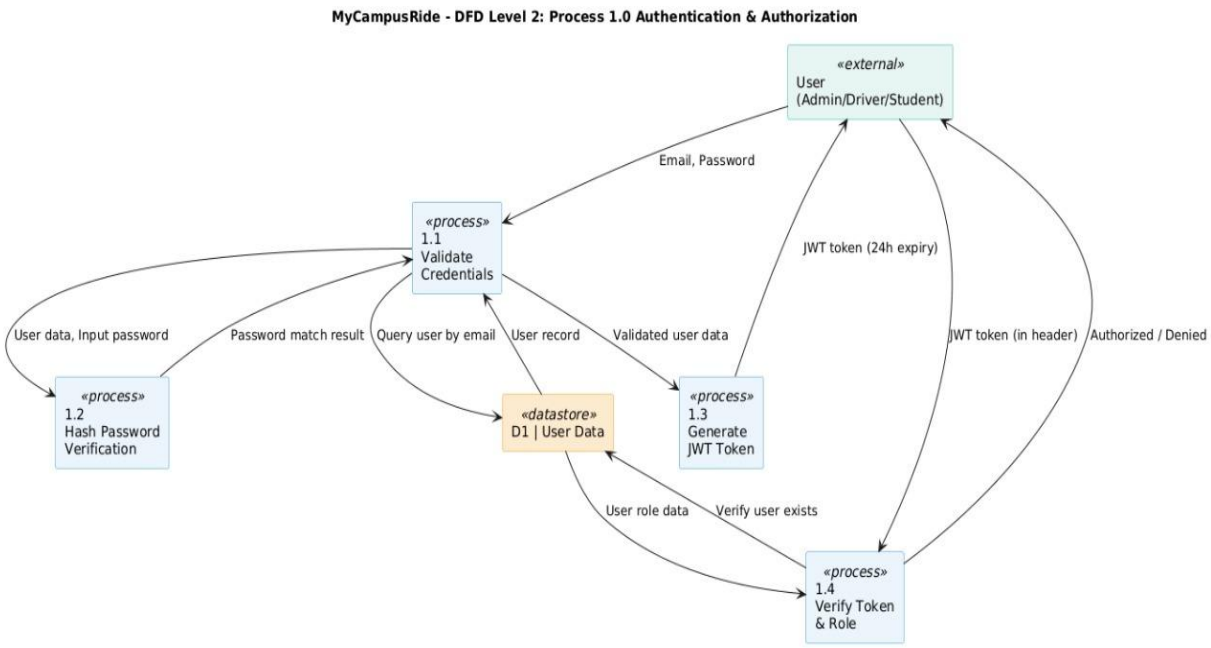
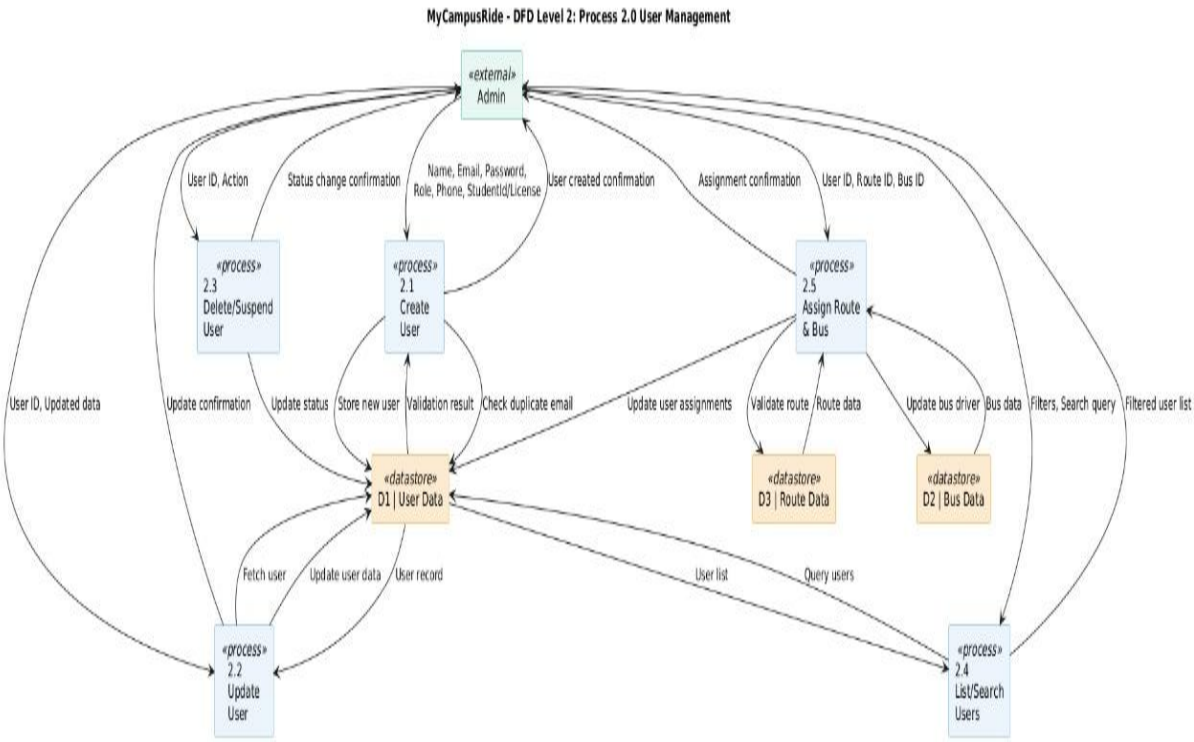


Figure 6.5: DFD Level 1

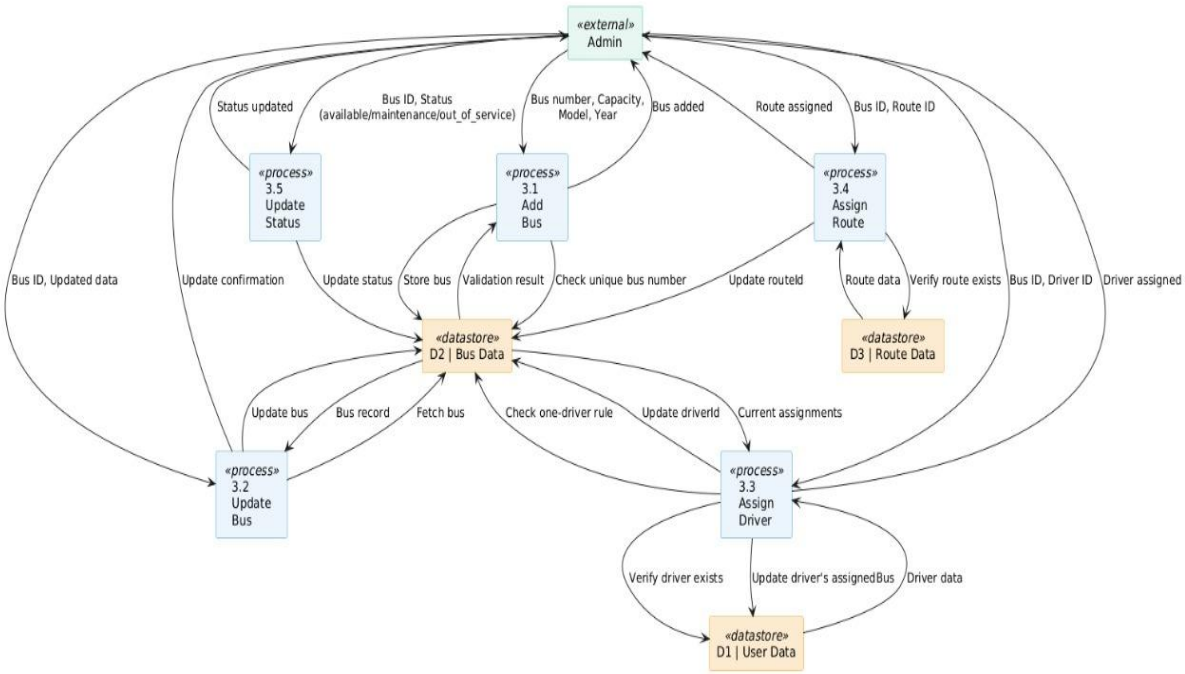
DFD Level 2 - Tracking Process:

This Tracking process is further decomposed into four subprocesses in the Level 2 DFD: Start Trip, Update Location, Get Bus Location, and Stop Trip. It also shows detailed data flows regarding GPS data, trip status updates, and location retrieval operations.



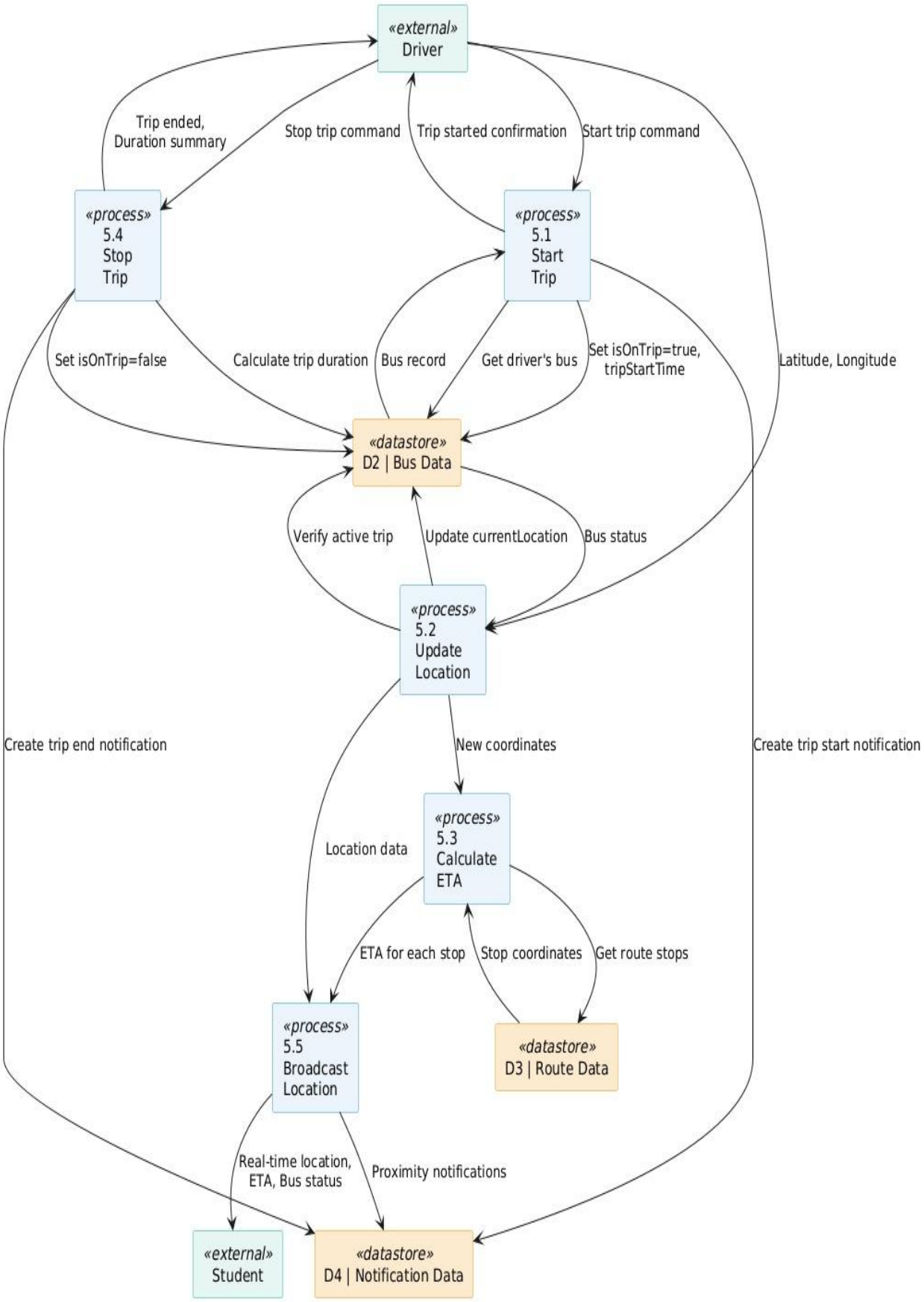


MyCampusRide - DFD Level 2: Process 3.0 Bus Management



MyCampusRide - DFD Level 2: Process 4.0 Route Management

MyCampusRide - DFD Level 2: Process 5.0 Real-time Tracking



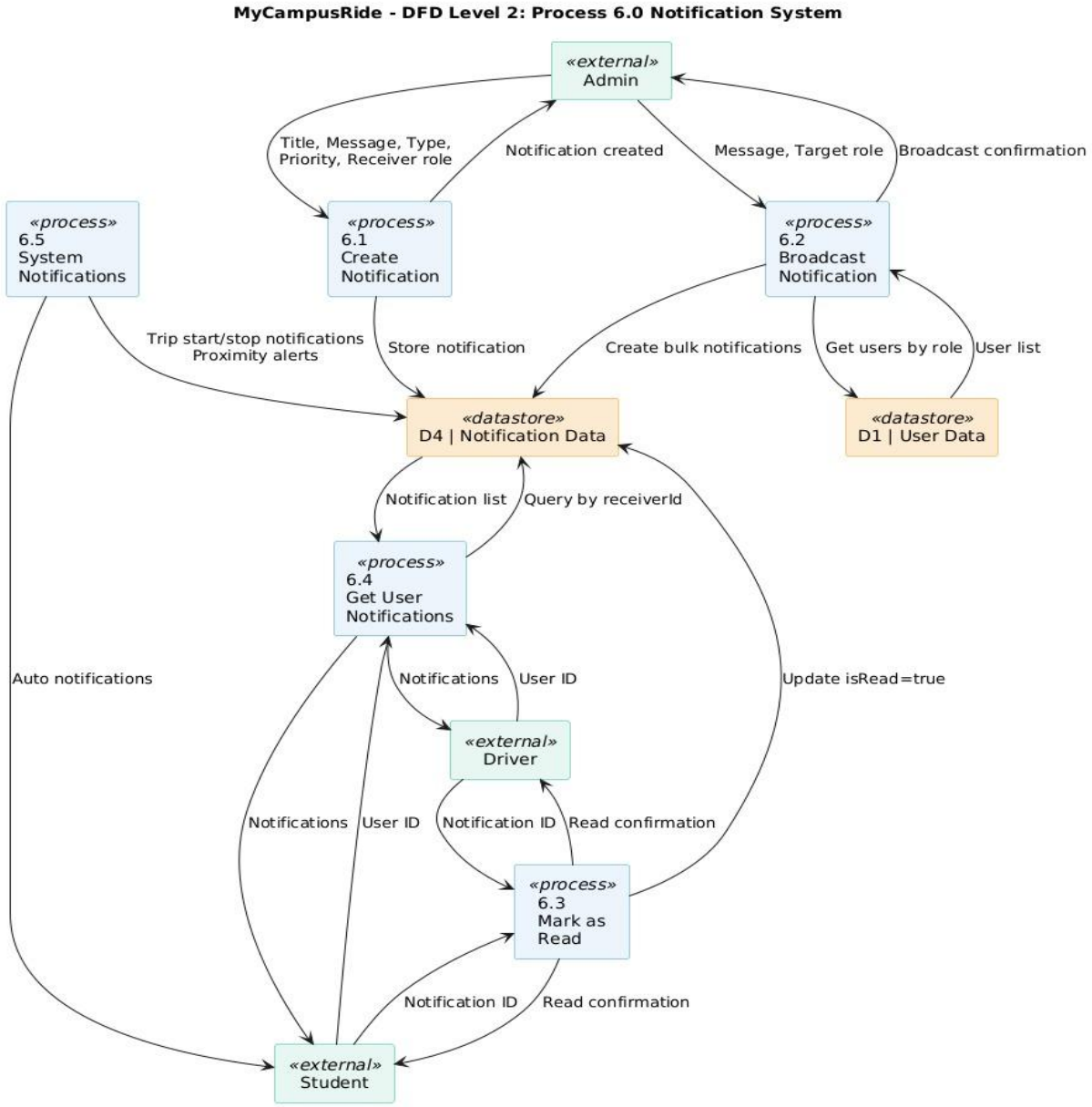


Figure 6.6: DFD Level 2 - Tracking Process

5. Data design

The MyCampusRide system uses MongoDB as its database management system. The data stored in document-oriented collections reflects the schema-on-read approach used by NoSQL databases, although some amount of structure is enforced through Mongoose schemas. The main four collections that the system operates under are Users, Buses, Routes, and Notifications. The Users collection stores information about all the users of the system, including authentication credentials, role-specific attributes, and resources assigned to them. The password is hashed by bcrypt before storing for security. Bus collection keeps track of information about buses, embedding location and referencing drivers and routes assigned. A Route collection would store route configurations with Stop subdocuments embedded for

flexible route management. Notification collection stores every notification of the system with sender and receiver information.

5.1 Data Dictionary

The following tables describe the data entities, their attributes, data types, and descriptions.

Table 7.1: User Entity

Attribute	Data Type	Description
_id	ObjectId	Unique identifier for the user
name	String	Full name of the user (2-50 characters)
email	String	Email address used for login (unique)
password	String	Hashed password using bcrypt
role	String	User role: admin, driver, or student
phone	String	Contact phone number
status	String	Account status: active, pending, suspended
studentId	String	Student identification number (students only)
feeStatus	String	Fee payment status: paid, partially_paid, pending
assignedBus	ObjectId	Reference to assigned bus
assignedRoute	ObjectId	Reference to assigned route
licenseNumber	String	Driver's license number (drivers only)

Table 7.2: Bus Entity

Attribute	Data Type	Description
_id	ObjectId	Unique identifier for the bus
busNumber	String	Bus identification number (unique)
driverId	ObjectId	Reference to assigned driver
routeId	ObjectId	Reference to assigned route
capacity	Number	Maximum passenger capacity (1-100)
currentLocation	Object	Current GPS coordinates and address
isOnTrip	Boolean	Whether bus is currently on a trip
status	String	Status: available, on_trip, maintenance, out_of_service
model	String	Bus model/make
year	Number	Manufacturing year
lastLocationUpdate	Date	Timestamp of last location update

Table 7.3: Route Entity

Attribute	Data Type	Description
_id	ObjectId	Unique identifier for the route
routeNo	String	Route number identifier (unique)
routeName	String	Descriptive route name (unique)
description	String	Route description (max 500 chars)
stops	Array	Array of Stop subdocuments
departureTime	String	Scheduled departure time (HH:MM)
distance	Number	Total route distance in kilometers
estimatedDuration	Number	Estimated trip duration in minutes
isActive	Boolean	Whether route is currently active
color	String	Display color (hex code)
assignedBuses	Array	Array of assigned bus ObjectIds

Table 7.4: Notification Entity

Attribute	Data Type	Description
_id	ObjectId	Unique identifier for the notification
title	String	Notification title (max 100 chars)
message	String	Notification message content (max 500 chars)
type	String	Type: info, warning, error, success, emergency
senderRole	String	Role of sender: admin, driver, system
senderId	ObjectId	Reference to sender user
receiverRole	String	Target role: admin, driver, student, all
receiverId	ObjectId	Reference to specific receiver
isRead	Boolean	Whether notification has been read
priority	String	Priority: low, medium, high, urgent

6. Algorithm & Implementation

This section describes the important algorithms implemented in the MyCampusRide system, using pseudocode representation. The algorithms describe critical operations such as authentication, managing trips, and handling notifications.

Algorithm 8.1: User Authentication

```
PROCEDURE authenticateUser(email, password) BEGIN     user ←
findUserByEmail(email)     IF user IS NULL THEN         RETURN
error("Invalid email or password")     END IF          isValid ←
comparePassword(password, user.hashedPassword)     IF NOT isValid THEN
RETURN error("Invalid email or password")     END IF          IF
user.status = "suspended" THEN         RETURN error("Account is suspended")
END IF          token ← generateJWT(user._id)     RETURN success(user,
token) END
```

Algorithm 8.2: Start Trip

```
PROCEDURE startTrip(driverId) BEGIN     bus ← findBusByDriver(driverId)
IF bus IS NULL THEN         RETURN error("No bus assigned")     END IF
IF bus.isOnTrip THEN         RETURN error("Trip already in progress")
END IF          IF bus.status ≠ "available" THEN         RETURN error("Bus
not available")     END IF          bus.isOnTrip ← TRUE     bus.status ←
"on_trip"     bus.tripStartTime ← currentDateTime()     saveBus(bus)
createNotification(         "Trip Started",         "Bus " + bus.busNumber
+ " has started",         "student"     )          RETURN success(bus) END
```

Algorithm 8.3: Update Bus Location

```
PROCEDURE updateLocation(driverId, latitude, longitude, address) BEGIN
bus ← findBusByDriver(driverId)     IF bus IS NULL THEN         RETURN
error("No bus assigned")     END IF          IF NOT bus.isOnTrip THEN
RETURN error("Not on trip")     END IF
bus.currentLocation.latitude ← latitude     bus.currentLocation.longitude ←
longitude     bus.currentLocation.address ← address
bus.lastLocationUpdate ← currentDateTime()     saveBus(bus)          RETURN
success(bus.currentLocation) END
```

Algorithm 8.4: Approve Driver

```
PROCEDURE approveDriver(driverId) BEGIN     driver ← findUserById(driverId)
IF driver IS NULL THEN         RETURN error("Driver not found")     END IF
IF driver.role ≠ "driver" THEN         RETURN error("User is not a driver")
END IF          IF driver.status ≠ "pending" THEN         RETURN
```

```
error("Driver not pending")     END IF          driver.status ← "active"
saveUser(driver)          createNotification(         "Account Approved",
"Your driver account has been approved",         "driver",
driver._id     )          RETURN success(driver) END
```

Algorithm 8.5: Create Route

```
PROCEDURE createRoute(routeData) BEGIN     IF routeExists(routeData.routeNo)
THEN         RETURN error("Route number exists")     END IF          IF
routeExists(routeData.routeName) THEN         RETURN error("Route name
exists")     END IF          IF routeData.stops.length > 0 THEN
sequences ← extractSequences(routeData.stops)         IF
hasDuplicates(sequences) THEN             RETURN error("Duplicate stop
sequences")         END IF     END IF          route ← new Route(routeData)
saveRoute(route)          RETURN success(route) END
```

7. Software requirements traceability matrix

A requirements traceability matrix links the functional requirements to design components and implementation elements. This provides full coverage of the requirements, while also making impact analysis of future changes much easier.

Req. No	Requirement	Design Component	Implementation
FR01	User Registration	Auth Module, User Model	authController.register()
FR02	User Authentication	Auth Module, JWT	authController.login()
FR03	Role-based Access	Middleware Layer	roleMiddleware.js
FR04	Bus Management	Bus Module, Bus Model	busController.js
FR05	Route Management	Route Module, Route Model	routeController.js
FR06	Real-time Tracking	Tracking Module	trackingController.js
FR07	Trip Start/Stop	Tracking Module	startTrip(), stopTrip()
FR08	Location Updates	Tracking Module	updateLocation()
FR09	Notifications	Notification Module	notificationController.js
FR10	Driver Approval	User Module	approveDriver()
FR11	User Statistics	User Module	getUserStats()
FR12	Bus Statistics	Bus Module	getBusStats()
FR13	Route Statistics	Route Module	getRouteStats()
FR14	Broadcast Messages	Notification Module	broadcastNotification()
FR15	Fee Management	User Model	feeStatus, feePaymentType

Table 9.1: Requirements Traceability Matrix

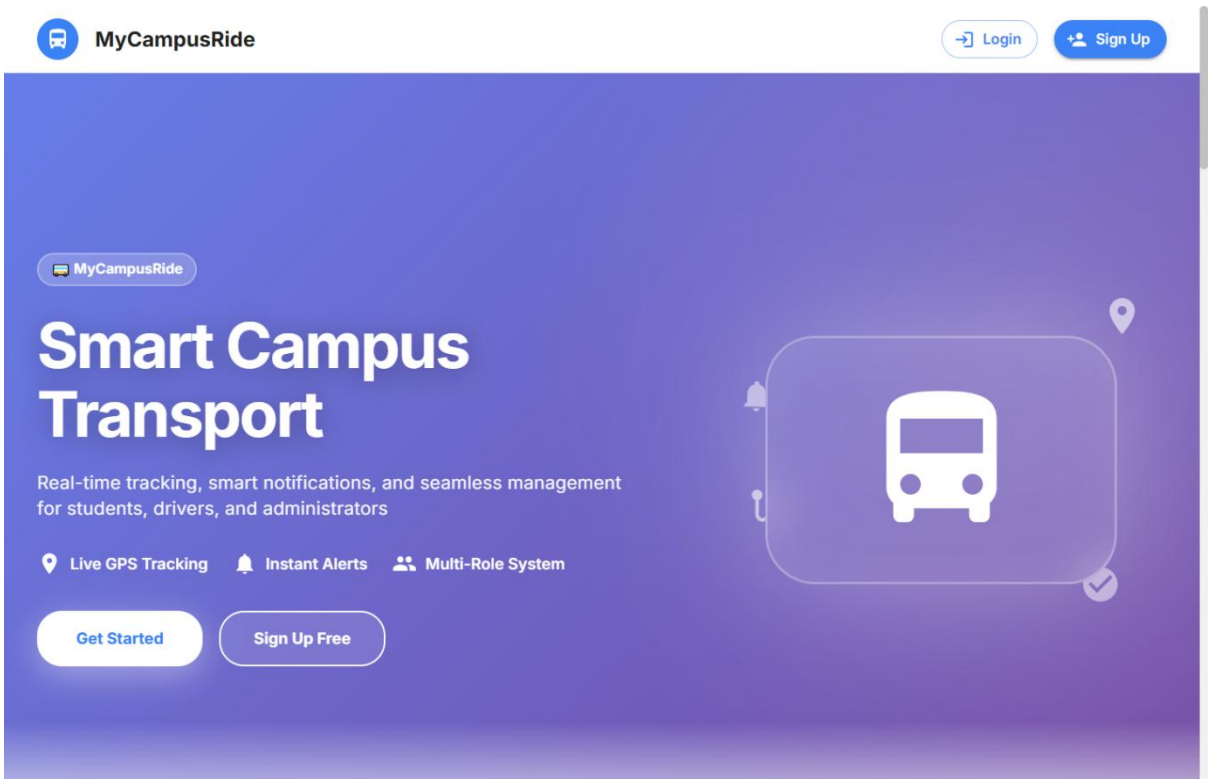
8. Human interface design

MyCampusRide provides a responsive web-based interface designed to be easy to operate and efficient for all user roles. The user interface is built using React.js with Material-UI components library, ensuring a consistent and modern look across all devices. The design follows established UI principles including clear visual hierarchy, eye-catchy color schemes, and responsive layouts working on desktop and mobile devices. The application will implement three role-based dashboards: Admin Dashboard, Driver Dashboard, and Student Dashboard. Each dashboard will offer tailored functionalities, depending on the user needs. Common elements include a navigation bar for user profile access, notification panel, and logout functionality. The landing page gives details about the service and provides links for access to the login form or registration form.

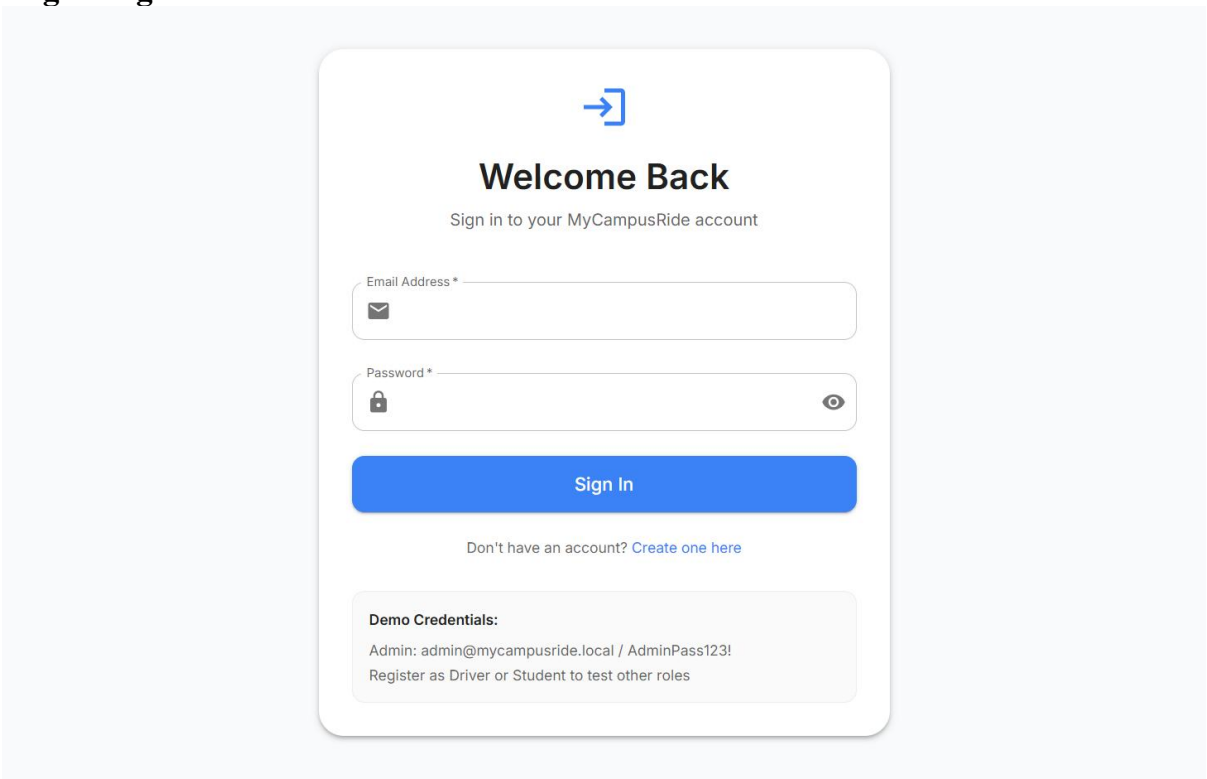
8.1 Screen images

The following describes the main screens and their purposes in the MyCampusRide system:

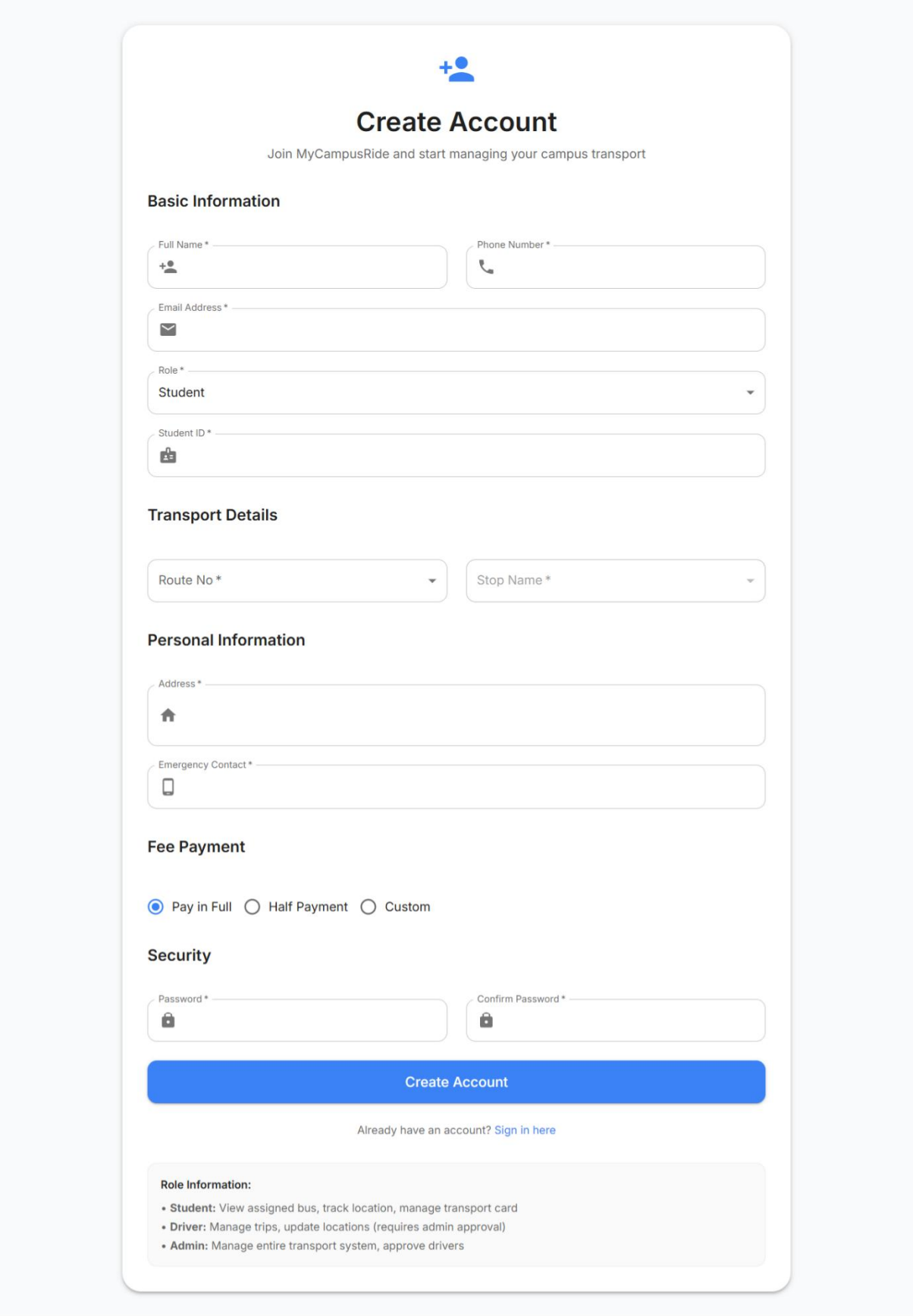
Landing Page:



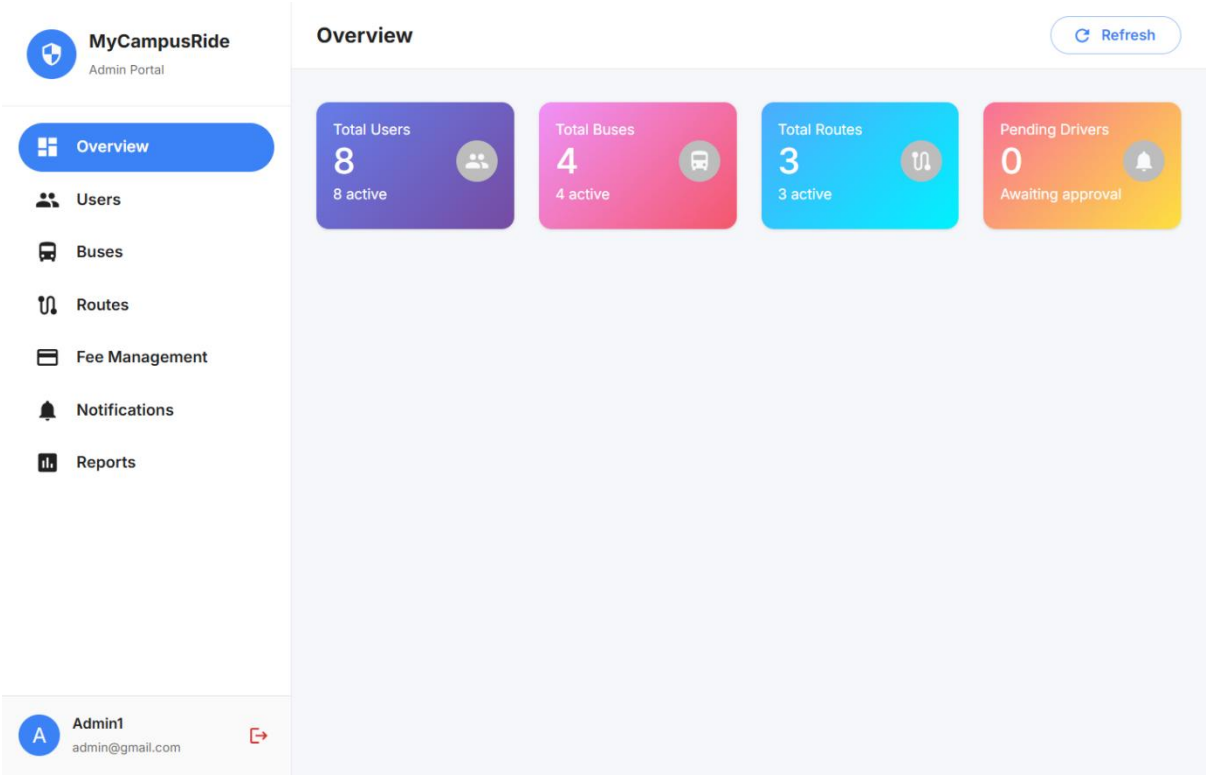
Login Page:

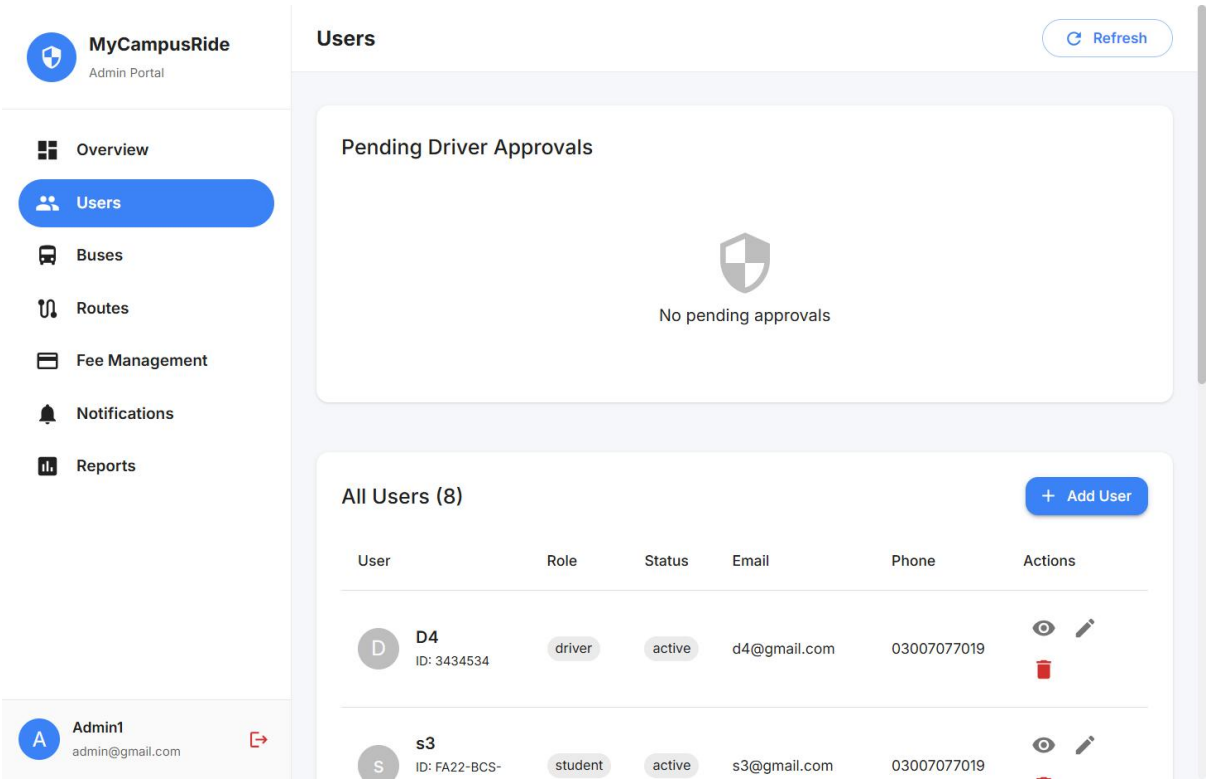


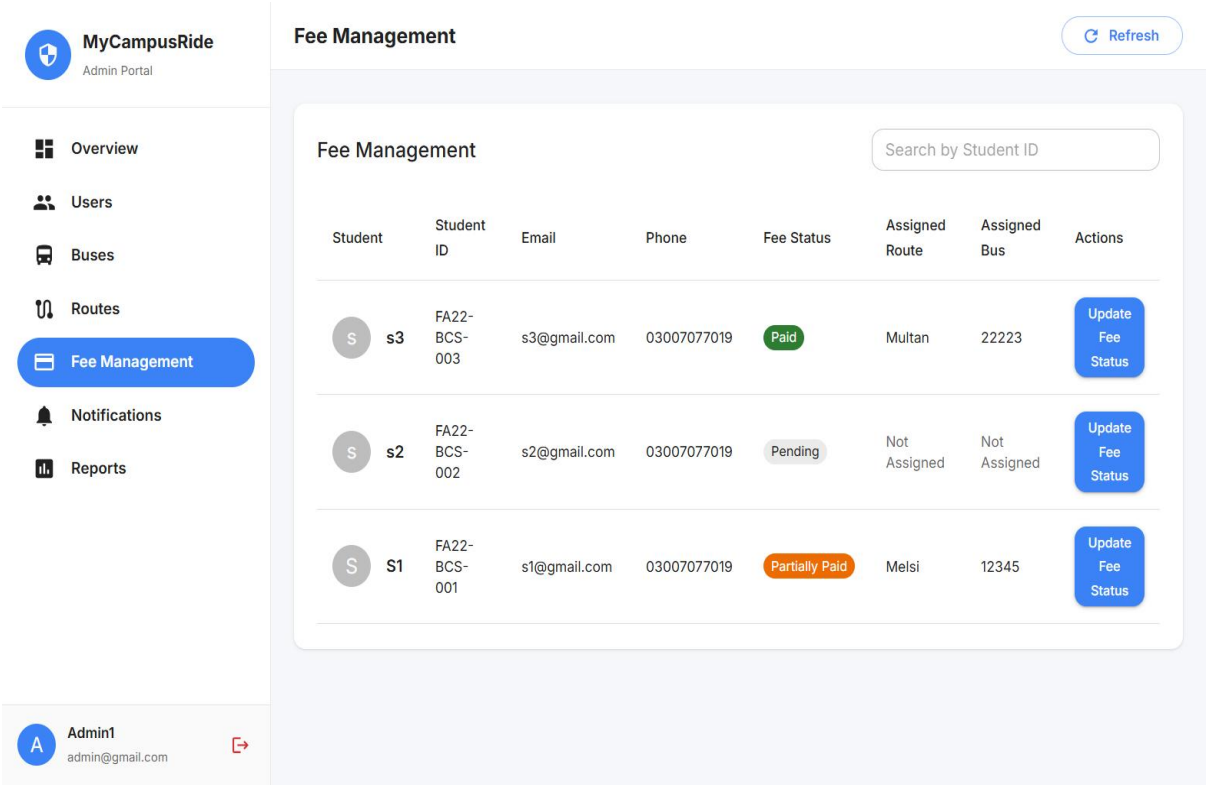
Registration Page:



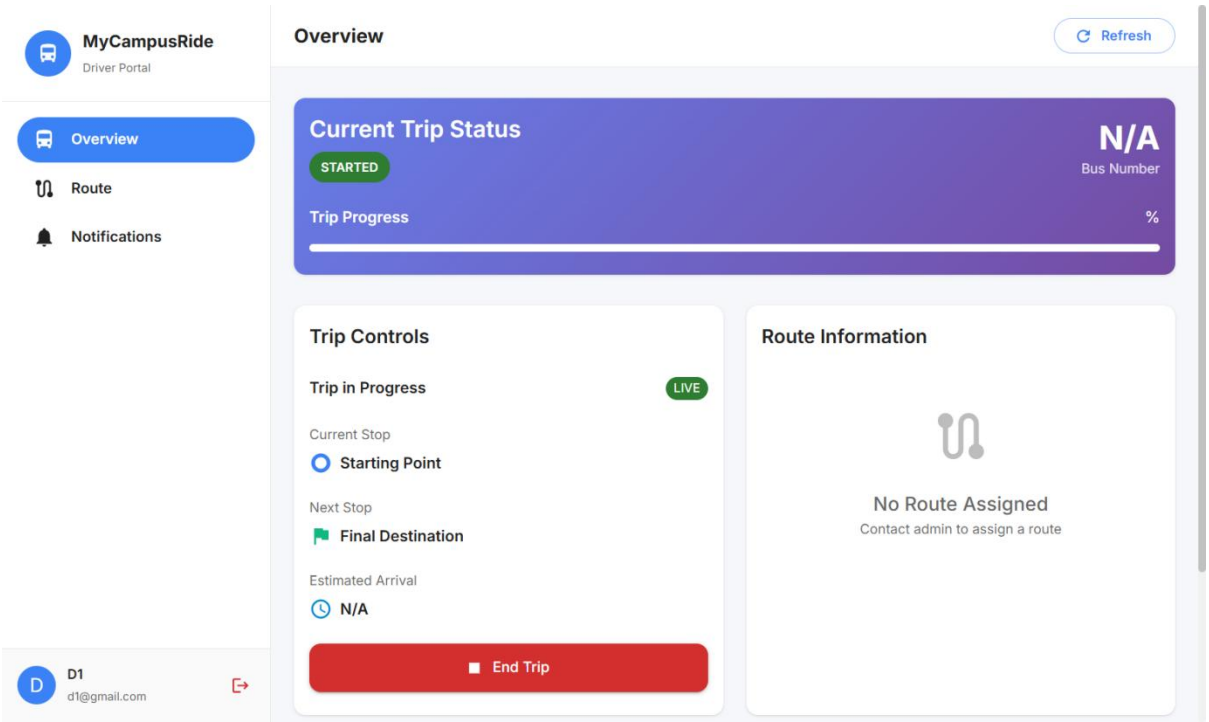
Admin Dashboard:



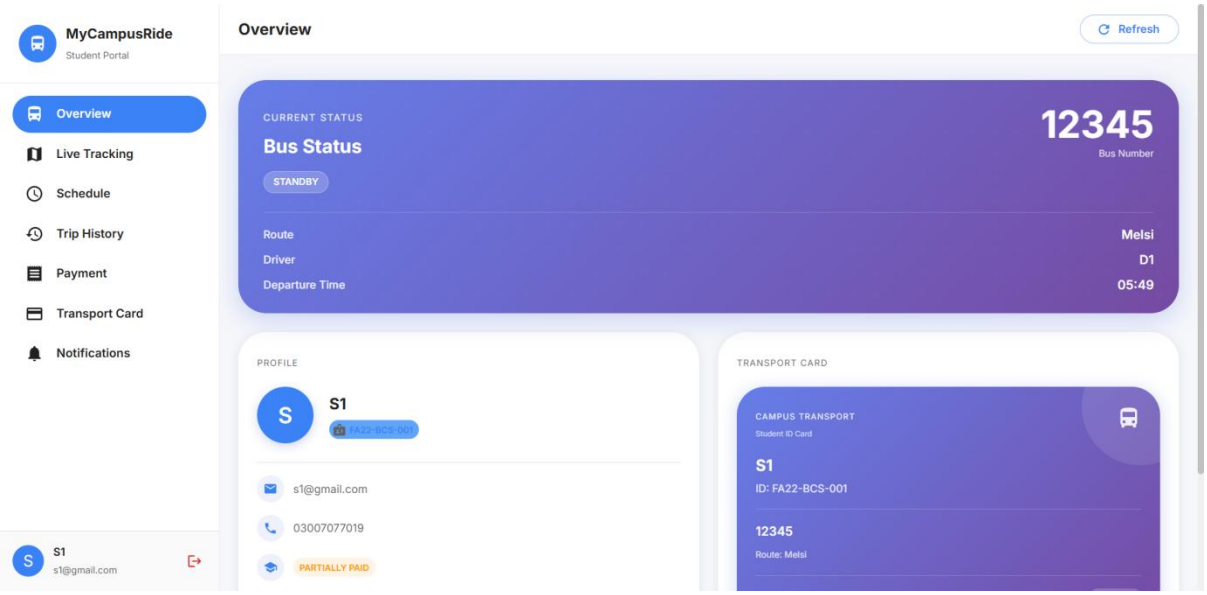


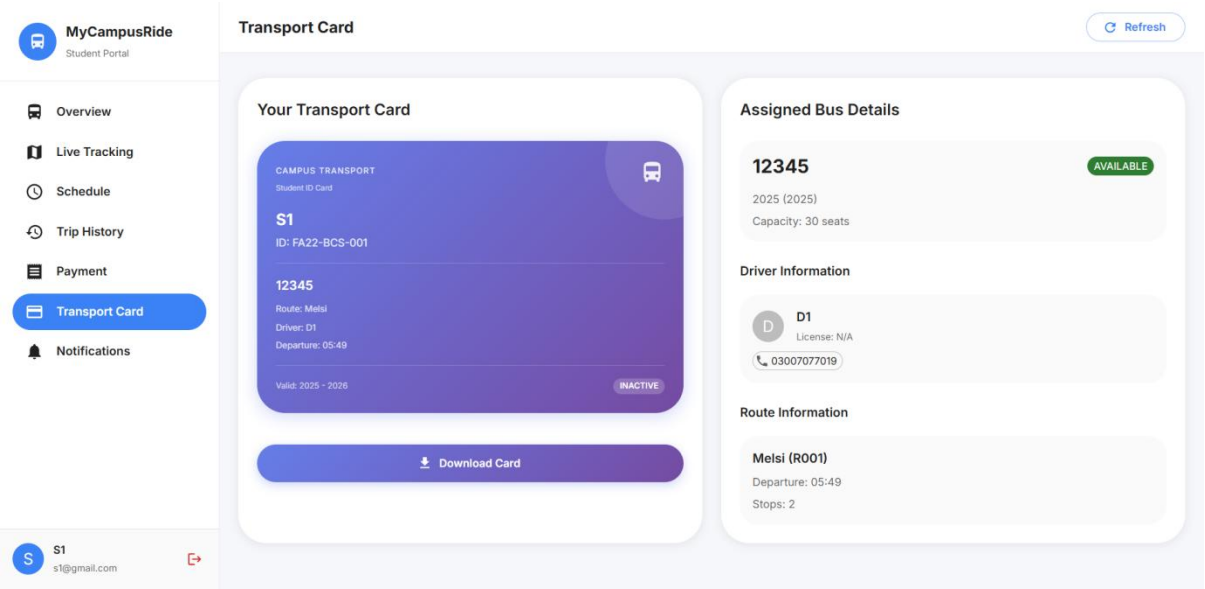


Driver Dashboard:



Student Dashboard:





8.2 Screen objects and actions

The following table describes the major screen objects and their associated actions:

Screen Object	Type	Actions
Navigation Bar	Component	Navigate to dashboard sections, access profile, logout
Login Form	Form	Enter credentials, submit for authentication
Registration Form	Form	Enter user details, select role, submit for account creation
User Table	Data Table	View users, filter by role/status, edit, delete, approve/reject drivers

Screen Object	Type	Actions
Bus Table	Data Table	View buses, filter by status, create, edit, delete buses
Route Table	Data Table	View routes, create, edit, delete, manage stops
Statistics Cards	Display	View counts of users, buses, routes by status
Start Trip Button	Button	Initiate bus trip, update status to on_trip
Stop Trip Button	Button	Complete bus trip, update status to available
Location Map	Map Component	Display bus location, route path, stops
Notification Panel	Panel	View notifications, mark as read, delete
Create Modal	Dialog	Enter entity details, validate, submit for creation
Edit Modal	Dialog	Modify entity details, validate, submit updates
Confirm Dialog	Dialog	Confirm or cancel destructive actions like delete
Filter Controls	Form Controls	Select filter criteria, apply to data tables

Table 10.1: Screen Objects and Actions

All interactive elements provide visual feedback through hover states, focus indicators, and loading states. Error handling is implemented throughout with user-friendly messages. The interface supports keyboard navigation for accessibility compliance.

References:

[1] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson, 2016.

[2] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed. New York, NY, USA: McGraw-Hill, 2015.

[3] K. Schwaber and J. Sutherland, "The Scrum Guide," Scrum.org, 2020. [Online]. Available: https://scrumguides.org

[4] Facebook Inc., "React – A JavaScript library for building user interfaces," 2024. [Online]. Available: https://react.dev

[5] Node.js Foundation, "Node.js Documentation," 2024. [Online]. Available: https://nodejs.org/en/docs

[6] Express.js, "Express – Fast, unopinionated, minimalist web framework for Node.js," 2024. [Online]. Available: https://expressjs.com

[7] MongoDB Inc., "MongoDB Manual," 2024. [Online]. Available: https://www.mongodb.com/docs

[8] Google Developers, "Material UI – React component library," 2024. [Online]. Available: https://mui.com

[9] Git SCM, "Git Documentation," 2024. [Online]. Available: https://git-scm.com/doc

[10] Auth0, "JSON Web Tokens Introduction," 2024. [Online]. Available: https://jwt.io/introduction